

Autodesk® **Scaleform**®

Scaleform 4.3 AS3

Autodesk® Scaleform® 4.3

© 2013 Autodesk, Inc. All rights reserved.

Disclaimer

1		1
2		3
3	DisplayObjectEx	9
4	FocusEventEx	12
5	FocusManager	13
6	GamePad	21
7	GamePadAnalogEvent	22
8	InteractiveObjectEx	25
9	KeyboardEventEx	28
10	MouseEvent	30
11	MouseEventEx	32
12	TextEventEx	35
13		37
14	TextFieldEx	38

1

```
scaleform.gfx.Extensions.enabled
```

```
scaleform.gfx.Extensions.enabled = true;
```

```
import scaleform.gfx.*;  
Extensions.enabled = true;
```


2

enabled

enabled:Boolean [read-write]

noInvisibleAdvance

noInvisibleAdvance:Boolean [read-write]

getTopMostEntity()

```
public function getTopMostEntity(x : Number, y : Number, testAll: Boolean) :  
DisplayObject
```

hitTes

getTopMostEntity

Extensions.getTopMostEntity(x, y).hitTest(x, y, true) == true

testAll : Boolean

getTopMostEntity

x : Number, y : Number

:

[getMouseTopMostEntity](#)

getMouseTopMostEntity()

```
public function getTopMostEntity([testAll : Boolean],  
    [mouseIndex : uint]) : DisplayObject
```

testAll : Boolean

mouseIndex : Number

:

[getTopMostEntity](#)

getMouseCursorType()

```
public function getMouseCursorType([mouseIndex : uint]) : String
```

- flash.ui.MouseCursor.ARROW : String = "arrow"
- flash.ui.MouseCursor.BUTTON : String = "button"
- flash.ui.MouseCursor.HAND : String = "hand"
- flash.ui.MouseCursor.IBEAM : String = "ibeam"

:

[setMouseCursorType](#)
[MouseEvent](#)

setMouseCursorType()

```
public function setMouseCursorType(cursor : String, [mouseIndex : uint]) : void
```

cursor : uint

- flash.ui.MouseCursor.ARROW : String = "arrow"
- flash.ui.MouseCursor.BUTTON : String = "button"
- flash.ui.MouseCursor.HAND : String = "hand"
- flash.ui.MouseCursor.IBEAM : String = "ibeam"

mouseIndex : Number

:

[getMouseEvent](#)
[MouseEvent](#)

numControllers

numControllers:uint [read]

setEdgeAAMode()

public function setEdgeAAMode(dispatcher:DisplayObject, mode:uint):void

•

•

-

-

dispObj : DisplayObject

mode : uint

:

[getEdgeAAMode](#)

getEdgeAAMode()

static public function getEdgeAAMode(dispObj:DisplayObject): uint

dispObj : DisplayObject

[setEdgeAAMode](#)

visibleRect

visibleRect:Rectangle [read]

isScaleform

isScaleform:Boolean [read]

3 DisplayObjectEx

setRendererString

```
static public function setRendererString(o:DisplayObject, s:String)
```

```
:
```

```
import scaleform.gfx.*;
```

```
// m is a MovieClip on stage.
```

```
DisplayObjectEx.setRendererString(m, "Abc");  
trace(DisplayObjectEx.getRendererString(m));
```

getRendererString()

```
static public function getRendererString(o:DisplayObject) : String
```

setRendererFloat

```
static public function setRendererFloat(o:DisplayObject, f:Number)
```

```
    :  
import scaleform.gfx.*;  
  
// m is a MovieClip on stage.  
  
DisplayObjectEx.setRendererFloat(m, 17.1717);  
trace(DisplayObjectEx.getRendererFloat(m));
```

getRendererFloat()

```
static public function getRendererFloat(o:DisplayObject) : Number
```

disableBatching

```
static public function disableBatching(o:DisplayObject, b:Boolean)
```

```
    :  
import scaleform.gfx.*;
```

```
// m is a MovieClip on stage.
```

```
DisplayObjectEx.disableBatching(batchDisable, true);  
trace(DisplayObjectEx.isBatchingDisabled(batchDisable));
```

isBatchingDisabled()

```
static public function isBatchingDisabled(o:DisplayObject) : Boolean
```

4 FocusEventEx

```
package scaleform.gfx
{
    import flash.events.FocusEvent;

    public final class FocusEventEx extends FocusEvent
    {
        public var controllerIdx : uint = 0;

        public function FocusEventEx(type:String) { super(type); }
    }
}
```

```
import scaleform.gfx.*;
import flash.events.FocusEvent;

Extensions.enabled = true;

function ev(e: FocusEvent)
{
    if (e is FocusEventEx)
    {
        var ee: FocusEventEx = e as FocusEventEx;
        trace("    controllerIdx = "+ee.controllerIdx);
    }
}

stage.addEventListener(FocusEvent.MOUSE_FOCUS_CHANGE, ev);
```

controllerIdx

```
controllerIdx : uint    [read]
```


5 FocusManager

alwaysEnableArrowKeys

`alwaysEnableArrowKeys: Boolean [read-write]`

```
_focusrect = false
```

```
alwaysEnableArrowKeys      true
```

disableFocusKeys

`disableFocusKeys: Boolean [read-write]`

moveFocus()

```
static public function moveFocus(keyToSimulate : String,  
                                startFromMovie:InteractiveObject = null,  
                                includeFocusEnabledChars : Boolean = false,  
                                controllerIdx : uint = 0) : InteractiveObject
```

disableFocusKeys modalClip

keyToSimulate : String

startFromMovie:InteractiveObject

includeFocusEnabledChars : Boolean

controllerIdx : uint

:

[findFocus](#)
[disableFocusKeys](#)
[setModalClip](#)
[getModalClip](#)

findFocus()

```
static public function findFocus(keyToSimulate : String,  
                                parentMovie:DisplayObjectContainer = null,  
                                loop : Boolean = false,  
                                startFromMovie:InteractiveObject = null,
```

```
includeFocusEnabledChars : Boolean = false,  
controllerIdx : uint = 0) : InteractiveObject
```

```
keyToSimulate : String
```

```
parentMovie:DisplayObjectContainer
```

```
loop : Boolean
```

```
startFromMovie:InteractiveObject
```

```
includeFocusEnabledChars : Boolean
```

```
controllerIdx : uint
```

```
:
```

```
moveFocus
```

```
disableFocusKeys
```

```
setModalClip
```

[getModalClip](#)

setFocus()

```
static public function setFocus(obj:InteractiveObject, controllerIdx:uint = 0) : void
```

obj:InteractiveObject

controllerIdx : uint

getFocus()

```
static public function getFocus(controllerIdx:uint = 0) : InteractiveObject
```

controllerIdx : uint

numFocusGroups

```
numFocusGroups() : uint [read]
```

setControllerFocusGroup

setFocusGroupMask()

public function setFocusGroupMask(obj:InteractiveObject, mask:uint) : void

```
FocusManager.setControllerFocusGroup(0, 0);  
FocusManager.setControllerFocusGroup(1, 1);  
  
FocusManager.setFocusGroupMask(button1, 0x1); // bit 0 - 0  
FocusManager.setFocusGroupMask(movieclip2, 0x1 | 0x2); // bits 0 and 1 -  
// 0 1
```

obj:InteractiveObject

mask:uint

:

[setControllerFocusGroup](#)
[getFocusGroupMask](#)

getFocusGroupMask()

```
static public function getFocusGroupMask(obj:InteractiveObject) : uint
```

obj:InteractiveObject

:

[setControllerFocusGroup](#)
[setFocusGroupMask](#)

setControllerFocusGroup()

```
static public function setControllerFocusGroup(controllerIdx:uint,  
                                              focusGroupId:uint) : Boolean
```

controllerIdx:uint

focusGroupId:uint

getControllerFocusGroup()

```
static public function getControllerFocusGroup(controllerIdx:uint) : uint
```

controllerIndex

setModalClip()

```
static public function setModalClip(mc:Sprite, controllerIdx:uint = 0) : void
```

controllerIdx:uint

mc:Sprite

6 GamePad

supportsAnalogEvents()

```
public static function supportsAnalogEvents() : Boolean
```

```
:
```

```
    GamePadAnalogEvent
```

7 GamePadAnalogEvent

```
package scaleform.gfx
{
    import flash.events.Event;

    public final class GamePadAnalogEvent extends Event
    {
        public static const CHANGE:String = "gamePadAnalogChange";

        public var code : uint = 0; // // scaleform.gfx.GamePad

        public var controllerIdx : uint = 0;
        public var xvalue : Number = 0; // [-1, 1]
        public var yvalue : Number = 0; // [-1, 1]

        public function GamePadAnalogEvent(bubbles:Boolean, cancelable:Boolean,
                                           code:uint, controllerIdx:uint = 0,
                                           xvalue:Number = 0, yvalue:Number = 0)
        {
            super(GamePadAnalogEvent.CHANGE, bubbles, cancelable);
            this.code = code;
            this.controllerIdx = controllerIdx;
            this.xvalue = xvalue;
            this.yvalue = yvalue;
        }
    }
}
```

```
import scaleform.gfx.*;

trace("GamePadAnalogEvents supported? " + GamePad.supportsAnalogEvents());
```

```
function ev(e: GamePadAnalogEvent)
{
    trace("code = " + ev.code);
    trace("controllerIdx = " + ev.controllerIdx);
    trace("xvalue = " + ev.xvalue);
    trace("yvalue = " + ev.yvalue);
}
stage.addEventListener(GamePadAnalogEvent.CHANGE, ev);
```

code : uint

controllerIdx

controllerIdx : uint

xvalue

xvalue : Number

yvalue

yvalue : Number

8 InteractiveObjectEx

getHitTestDisable()

```
public function getHitTestDisable(o:InteractiveObject) : Boolean
```

:

[InteractiveObjectEx.setHitTestDisable](#)

setHitTestDisable()

```
public function setHitTestDisable(o:InteractiveObject, f:Boolean) : void
```

```
o
f
:
InteractiveObjectEx.getHitTestDisable
```

getTopmostLevel()

```
public function getTopmostLevel (o:InteractiveObject) : Boolean
```

[InteractiveObjectEx.setTopmostLevel](#)

setTopmostLevel ()

```
public function setTopmostLevel(o:InteractiveObject, f:Boolean) : void
```

o

f

9 KeyboardEventEx

```
package scaleform.gfx
{
    import flash.events.KeyboardEvent;

    public final class KeyboardEventEx extends KeyboardEvent
    {
        public var controllerIdx : uint = 0;

        public function KeyboardEventEx(type:String) { super(type); }
    }
}
```

```
import scaleform.gfx.*;
import flash.events.KeyboardEvent;

Extensions.enabled = true;

function ev(e: KeyboardEvent)
{
    if (e is KeyboardEventEx)
    {
        var ee: KeyboardEventEx = e as KeyboardEventEx;
        trace("    controllerIdx = "+ee.controllerIdx);
    }
}
stage.addEventListener(KeyboardEvent.KEY_DOWN, ev);
stage.addEventListener(KeyboardEvent.KEY_UP, ev);
```

controllerIdx

```
controllerIdx : uint [read]
```


10 MouseCursorEvent

```
package scaleform.gfx
{
    import flash.events.Event;

    public final class MouseCursorEvent extends Event
    {
        public var cursor : String = "auto";
        public var mouseIdx : uint = 0;

        static public const CURSOR_CHANGE : String = "mouseCursorChange";

        public function MouseCursorEvent()
        {
            super("MouseCursorEvent", false, true);
        }
    }
}
```

bubbles

cancellable

flash.ui.Mouse.cursor

Extensions.enabled = true;

```
function e(e:MouseCursorEvent)
{
    trace(e.type + " " + e.mouseIdx + " " + e.cursor);
    e.preventDefault();
}
stage.addEventListener(MouseCursorEvent.CURSOR_CHANGE, e);
```

`cursor : String [read]`

- `flash.ui.MouseCursor.ARROW : String = "arrow"`
- `flash.ui.MouseCursor.BUTTON : String = "button"`
- `flash.ui.MouseCursor.HAND : String = "hand"`
- `flash.ui.MouseCursor.IBEAM : String = "ibeam"`

mouseIdx

`mouseIdx : uint [read]`

11 MouseEventEx

```
package scaleform.gfx
{
    import flash.events.MouseEvent;

    public final class MouseEventEx extends MouseEvent
    {
        public var mouseIdx : uint = 0;
        public var nestingIdx : uint = 0;
        public var buttonIdx : uint = 0; // LEFT_BUTTON, RIGHT_BUTTON, ...

        public static const LEFT_BUTTON : uint = 0;
        public static const RIGHT_BUTTON : uint = 1;
        public static const MIDDLE_BUTTON : uint = 2;

        public function MouseEventEx(type:String) { super(type); }
    }
}
```

```
import scaleform.gfx.*;
Extensions.enabled = true;

stage.doubleClickEnabled = true;

function ev(e:MouseEvent):void
{
    trace("!!!! EVENT. " + cnt++);
    trace("    eventType      = "+e.type);
    trace("    bubbles          = "+e.bubbles);
    trace("    eventPhase        = "+e.eventPhase);
    trace("    target            = "+e.target.name);
    trace("    currentTarget     = "+e.currentTarget.name);
    if (e is MouseEventEx)
    {
        var ee:MouseEventEx = e as MouseEventEx;
```

```

        trace("    mouseIdx      = "+ee.mouseIdx);
        trace("    nestingIdx    = "+ee.nestingIdx);
        trace("    buttonIdx     = "+ee.buttonIdx);
    }
    trace(e);
}
stage.addEventListener(MouseEvent.CLICK, ev);
stage.addEventListener(MouseEvent.DRAG_START, ev);
stage.addEventListener(MouseEvent.DRAG_OVER, ev);
stage.addEventListener(MouseEvent.DRAG_END, ev);
stage.addEventListener(MouseEvent.DOUBLE_CLICK, ev);

```

buttonIdx

buttonIdx : uint [read]

-
-
-
-

mouseIdx

mouseIdx : uint [read]

nestingIdx

nestingIdx : uint [read]

12 TextEventEx

```
package scaleform.gfx
{
    import flash.events.TextEvent;

    public final class TextEventEx extends TextEvent
    {
        public var controllerIdx : uint = 0;

        public function TextEventEx(type:String) { super(type); }
    }
}
```

```
import scaleform.gfx.*;
import flash.events.TextEvent;

Extensions.enabled = true;

function ev(e: TextEvent)
{
    if (e is TextEventEx)
    {
        var ee: TextEventEx = e as TextEventEx;
        trace("    controllerIdx = "+ee.controllerIdx);
    }
}

txf.addEventListener(TextEvent.TEXT_INPUT, ev);
```

controllerIdx

```
controllerIdx : uint    [read]
```

LINK_MOUSE_OVER/LINK_MOUSE_OUT

```
public static const LINK_MOUSE_OVER:String = "linkMouseOver";  
public static const LINK_MOUSE_OUT:String = "linkMouse";
```

buttonIdx

```
buttonIdx : uint [read]
```


13

actionVerbose

`actionVerbose:Boolean` [read-write]

getStackTrace()

`public function getStackTrace() : String`

getCodeFileName()

`public function getCodeFileName() : String`

14 TextFieldEx

appendHtml()

```
public function appendHtml(textField:TextField, newHtml:String) : void
```

```
txt.htmlText += moreHtml
```

```
textField:TextField -  
newHtml:String
```

setIMEEnabled()

```
static public function setIMEEnabled(textField:TextField, isEnabled:Boolean): void
```

```
textField : TextField  
isEnabled : Boolean true
```

setVerticalAlign()

```
public function setVerticalAlign(textField:TextField, valign:String) : void
```

```
public static const VALIGN_TOP:String      = "top";  
public static const VALIGN_CENTER:String   = "center";  
public static const VALIGN_BOTTOM:String   = "bottom";
```



```
textField:TextField  
valign:String      ("top", "center", "bottom").
```

getVerticalAlign()

```
static public function getVerticalAlign(textField:TextField) : String
```

setImageSubstitutions ()

```
public setImageSubstitutions(substInfoArr:Array) : void  
public setImageSubstitutions(substInfo:Object) : void  
public setImageSubstitutions(null) : void
```

BitmapData

```
import flash.display.BitmapData;  
var imageBmp:BitmapData = BitmapData.loadBitmap("myImage");
```

```
: import flash.display.BitmapData;
```

flash.display.BitmapData

subString:String

image : BitmapData

width : Number

height : Number

baseLineY : Number

id : String

"substInfoArr"

"substInfo"

setImageSubstitutions(substInfo:Object)

setImageSubstitutions(substInfoArr:Array)

setImageSubstitutions

setImageSubstitutions(null)

setImageSubstitutions

Parameters

substInfoArr:Array -

substInfo:Object -

null -

updateImageSubstitution()

```
var b1 = BitmapData.loadBitmap("smile1");
var b2 = BitmapData.loadBitmap("smile2");
var b3 = BitmapData.loadBitmap("smile3");
var a = new Array;
a[0] = { subString:"=", image:b1, baseLineY:35, width:20, height:20, id:"sm=" } };
a[1] = { subString:":-)", image:b2, baseLineY:20, id:"sm:-)" } };
a[2] = { subString:":\\", image:b3, baseLineY:35, height:100 } };
a[3] = { subString:"-\\", image:b1 } };
t.setImageSubstitutions(a);
```

updateImageSubstitution ()

```
public updateImageSubstitution(id:String, image:BitmapData) : void
```

setImageSubstitutions

Parameters

id:String -

setImageSubstitutions

```
image:BitmapData -
```

```
setImageSubstitutions()
```

```
t.updateImageSubstitution("sm=", b3);
```

```
updateImageSubstitution
```

```
var phase = 0;
var b1a = BitmapData.loadBitmap("smile1a");
var b2a = BitmapData.loadBitmap("smile2a");

onEnterFrame = function()
{
    if (phase % 10 == 0)
    {
        if (phase % 20 == 0)
        {
            _root.t.updateImageSubstitution("sm=", b1);
            _root.t.updateImageSubstitution("sm:-)", b2);
        }
        else
        {
            _root.t.updateImageSubstitution("sm=", b1a);
            _root.t.updateImageSubstitution("sm:-)", b2a);
        }
    }
    ++phase;
}
```

setTextAutoSize() static method

```
static public function setTextAutoSize(textField:TextField, autoSz:String) : void
```

```

public static const TEXTAUTOSZ_NONE:String      = "none";
public static const TEXTAUTOSZ_SHRINK:String    = "shrink";
public static const TEXTAUTOSZ_FIT:String       = "fit";

```

```

fit
shrink"

```



```

textField:TextField -
autoSz:String       -

```

getTextAutoSize() static method

```

static public function getTextAutoSize(textField:TextField) : String

```

setNoTranslate() static method


```
static public function setNoTranslate(textField:TextField, noTranslate:Boolean) :  
                                void
```

Scaleform version: 4.1.20

getNoTranslate() static method

```
static public function getNoTranslate(textField:TextField) : Boolean
```

Scaleform version: 4.1.20